01_corpus_descriptive_report

May 8, 2026

1 01 Corpus Descriptive Report

This notebook is a descriptive reporting layer for the AfD project post-retrieval data.

Scope: - Uses existing `data/` tables only (no PDF parsing/retrieval/cleaning/dedup reruns) - Uses manual monthly Nexis totals as the **main** media salience indicator - Treats downloaded and cleaned article counts as **diagnostic sample-size series**

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "data").exists() and (PROJECT_ROOT.parent / "data").
 ↪exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / "data"
FIGURES_DIR = PROJECT_ROOT / "figures" / "01_notebook_figures"
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

MONTHLY_SUMMARY_PATH = DATA_DIR / "corpus" / "monthly_summary.csv"
NEXIS_VOLUME_PATH = DATA_DIR / "corpus" / "monthly_nexis_volume.csv"

SALIENCE_COL = "nexis_total_results"
CLEANED_SAMPLE_COL = "kept_article_count"
DOWNLOAD_COL = "raw_article_count"

print("PROJECT_ROOT:", PROJECT_ROOT)
print("MONTHLY_SUMMARY_PATH:", MONTHLY_SUMMARY_PATH)
print("NEXIS_VOLUME_PATH:", NEXIS_VOLUME_PATH)
print("FIGURES_DIR:", FIGURES_DIR)
```

```
PROJECT_ROOT: c:\PythonProjects\AfD-project
MONTHLY_SUMMARY_PATH: c:\PythonProjects\AfD-
project\data\corpus\monthly_summary.csv
NEXIS_VOLUME_PATH: c:\PythonProjects\AfD-
```

```
project\data\corpus\monthly_nexis_volume.csv
FIGURES_DIR: c:\PythonProjects\AfD-project\figures\01_notebook_figures
```

[2]:
```python
monthly = pd.read_csv(MONTHLY_SUMMARY_PATH)
nexis_volume = pd.read_csv(NEXIS_VOLUME_PATH)

print("monthly shape:", monthly.shape)
print("nexis_volume shape:", nexis_volume.shape)
print("monthly columns:", list(monthly.columns))
print("nexis_volume columns:", list(nexis_volume.columns))
```

```
monthly shape: (156, 32)
nexis_volume shape: (156, 3)
monthly columns: ['month_id', 'year', 'month', 'batch_id', 'raw_article_count',
'kept_article_count', 'dropped_article_count', 'keep_share',
'unique_source_count', 'total_word_count_kept', 'mean_word_count_kept',
'median_word_count_kept', 'duplicate_exact_count', 'duplicate_near_count',
'regional_variant_count', 'malformed_count', 'reader_letter_drop_count',
'commentary_noncore_drop_count', 'very_short_low_value_drop_count',
'source_top1', 'source_top1_count', 'source_top2', 'source_top2_count',
'source_top3', 'source_top3_count', 'freeze_status', 'notes',
'kept_share_of_raw', 'duplicate_burden', 'dropped_share', 'shock_month_flag',
'shock_window_flag_placeholder']
nexis_volume columns: ['month_id', 'volume', 'raw_article_count']
```

[3]:
```python
expected_months = pd.period_range("2013-01", "2025-12", freq="M").astype(str)
expected_months_set = set(expected_months)

required_nexis_cols = {"month_id", "volume", "raw_article_count"}
missing_nexis_cols = required_nexis_cols - set(nexis_volume.columns)
assert not missing_nexis_cols, f"Missing required NEXIS columns:␣
 ↪{missing_nexis_cols}"

assert len(nexis_volume) == 156, f"Expected 156 rows in monthly_nexis_volume.
 ↪csv, found {len(nexis_volume)}"
assert not nexis_volume["month_id"].duplicated().any(), "Duplicate month_id in␣
 ↪monthly_nexis_volume.csv"

found_months = set(nexis_volume["month_id"].astype(str))
missing_months = sorted(expected_months_set - found_months)
extra_months = sorted(found_months - expected_months_set)
assert not missing_months, f"Missing months in NEXIS volume file:␣
 ↪{missing_months}"
assert not extra_months, f"Unexpected extra months in NEXIS volume file:␣
 ↪{extra_months}"

for col in ["volume", "raw_article_count"]:
    nexis_volume[col] = pd.to_numeric(nexis_volume[col], errors="coerce")
```

```
    assert nexis_volume[col].notna().all(), f"Column {col} has non-numeric or␣
 ↪missing values"

assert len(monthly) == 156, f"Expected 156 rows in monthly_summary, found␣
 ↪{len(monthly)}"
assert "month_id" in monthly.columns, "monthly_summary must include month_id"
assert not monthly["month_id"].duplicated().any(), "Duplicate month_id in␣
 ↪monthly_summary"

print("Validation passed: monthly_nexis_volume and monthly_summary structural␣
 ↪checks are clean.")
```

```
Validation passed: monthly_nexis_volume and monthly_summary structural checks
are clean.
```

[4]:
```
monthly2 = monthly.copy()
nexis2 = nexis_volume.rename(columns={"volume": "nexis_total_results",␣
 ↪"raw_article_count": "raw_article_count_manual"}).copy()

if "raw_article_count" in monthly2.columns:
    monthly2 = monthly2.rename(columns={"raw_article_count":␣
 ↪"raw_article_count_monthly"})
else:
    monthly2["raw_article_count_monthly"] = np.nan

vol = monthly2.merge(nexis2, on="month_id", how="left", validate="one_to_one")
vol["month_dt"] = pd.to_datetime(vol["month_id"] + "-01", errors="coerce")

vol["raw_article_count_mismatch"] = (
    vol["raw_article_count_monthly"].notna()
    & vol["raw_article_count_manual"].notna()
    & (vol["raw_article_count_monthly"] != vol["raw_article_count_manual"])
)

vol["raw_article_count"] = vol["raw_article_count_manual"].where(
    vol["raw_article_count_manual"].notna(), vol["raw_article_count_monthly"]
)

mismatch_rows = vol.loc[
    vol["raw_article_count_mismatch"],
    ["month_id", "raw_article_count_monthly", "raw_article_count_manual"],
].sort_values("month_id")

print("Merged rows:", len(vol))
print("Missing nexis_total_results:", int(vol["nexis_total_results"].isna().
 ↪sum()))
print("Raw article count mismatches:", len(mismatch_rows))
```

```
if len(mismatch_rows):
    display(mismatch_rows)

display(vol[["month_id", "nexis_total_results", "raw_article_count",␣
 ↪"kept_article_count"]].head(12))
```

```
Merged rows: 156
Missing nexis_total_results: 0
Raw article count mismatches: 0
```

	month_id	nexis_total_results	raw_article_count	kept_article_count
0	2013-01	42	42	38
1	2013-02	55	55	41
2	2013-03	67	67	61
3	2013-04	63	63	55
4	2013-05	64	64	54
5	2013-06	83	83	71
6	2013-07	104	104	91
7	2013-08	146	146	130
8	2013-09	95	91	80
9	2013-10	151	141	125
10	2013-11	146	136	117
11	2013-12	131	50	45

[5]:
```
# z-standardization uses population SD (ddof=0) for consistency with project␣
 ↪outputs.
vol["media_salience_volume_raw"] = vol[SALIENCE_COL]
vol["media_salience_volume_log1p"] = np.log1p(vol[SALIENCE_COL])

sal_mean = vol[SALIENCE_COL].mean()
sal_std = vol[SALIENCE_COL].std(ddof=0)
vol["media_salience_volume_z"] = (vol[SALIENCE_COL] - sal_mean) / sal_std if pd.
 ↪notna(sal_std) and sal_std != 0 else np.nan

vol["cleaned_sample_size"] = pd.to_numeric(vol[CLEANED_SAMPLE_COL],␣
 ↪errors="coerce")
vol["cleaned_sample_log1p"] = np.log1p(vol["cleaned_sample_size"])
clean_mean = vol["cleaned_sample_size"].mean()
clean_std = vol["cleaned_sample_size"].std(ddof=0)
vol["cleaned_sample_z"] = (vol["cleaned_sample_size"] - clean_mean) / clean_std␣
 ↪if pd.notna(clean_std) and clean_std != 0 else np.nan

valid_den = vol[SALIENCE_COL].notna() & (vol[SALIENCE_COL] > 0)
vol["download_fraction"] = np.where(valid_den, vol[DOWNLOAD_COL] /␣
 ↪vol[SALIENCE_COL], np.nan)
vol["cleaned_sample_fraction"] = np.where(valid_den, vol[CLEANED_SAMPLE_COL] /␣
 ↪vol[SALIENCE_COL], np.nan)
```

```
vol["retrieval_mismatch_flag"] = vol[DOWNLOAD_COL] > vol[SALIENCE_COL]
vol["high_download_sample_flag"] = vol[DOWNLOAD_COL] > 50

display(vol[[
    "month_id", "nexis_total_results", "raw_article_count",␣
 ↪"kept_article_count",
    "download_fraction", "cleaned_sample_fraction", "retrieval_mismatch_flag",␣
 ↪"high_download_sample_flag"
]].head(12))
```

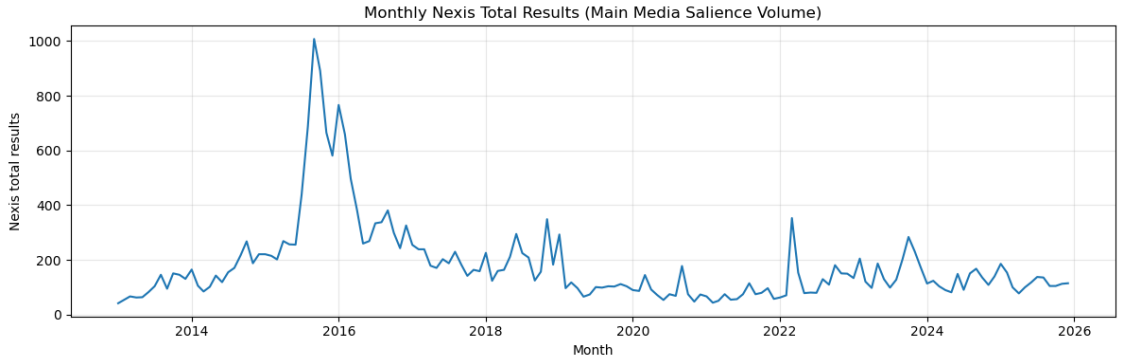
	month_id	nexis_total_results	raw_article_count	kept_article_count	\
0	2013-01	42	42	38	
1	2013-02	55	55	41	
2	2013-03	67	67	61	
3	2013-04	63	63	55	
4	2013-05	64	64	54	
5	2013-06	83	83	71	
6	2013-07	104	104	91	
7	2013-08	146	146	130	
8	2013-09	95	91	80	
9	2013-10	151	141	125	
10	2013-11	146	136	117	
11	2013-12	131	50	45	

	download_fraction	cleaned_sample_fraction	retrieval_mismatch_flag	\
0	1.000000	0.904762	False	
1	1.000000	0.745455	False	
2	1.000000	0.910448	False	
3	1.000000	0.873016	False	
4	1.000000	0.843750	False	
5	1.000000	0.855422	False	
6	1.000000	0.875000	False	
7	1.000000	0.890411	False	
8	0.957895	0.842105	False	
9	0.933775	0.827815	False	
10	0.931507	0.801370	False	
11	0.381679	0.343511	False	

	high_download_sample_flag
0	False
1	True
2	True
3	True
4	True
5	True
6	True
7	True

```
8                          True
9                          True
10                         True
11                        False
```

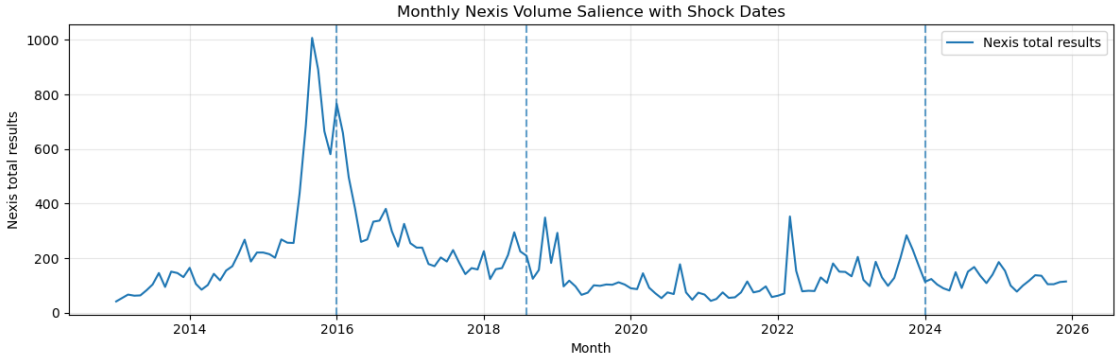
[6]:
```
plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol["nexis_total_results"], linewidth=1.5)
plt.title("Monthly Nexis Total Results (Main Media Salience Volume)")
plt.xlabel("Month")
plt.ylabel("Nexis total results")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(FIGURES_DIR / "monthly_nexis_total_results_over_time_report.png",␣
 ↪dpi=150)
plt.show()
print("Saved:", FIGURES_DIR / "monthly_nexis_total_results_over_time_report.
 ↪png")
```




```
Saved: c:\PythonProjects\AfD-project\figures\01_notebook_figures\monthly_nexis_t
otal_results_over_time_report.png
```

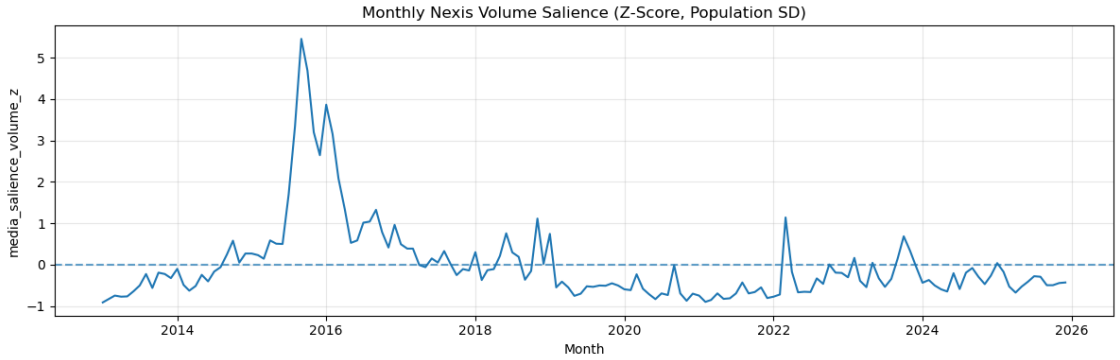
[7]:
```
shock_dates = pd.to_datetime(["2016-01-01", "2018-08-01", "2024-01-01"])

plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol["nexis_total_results"], linewidth=1.5,␣
 ↪label="Nexis total results")
for d in shock_dates:
    plt.axvline(d, linestyle="--", alpha=0.7)
plt.title("Monthly Nexis Volume Salience with Shock Dates")
plt.xlabel("Month")
plt.ylabel("Nexis total results")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
```

```
plt.savefig(FIGURES_DIR / "monthly_volume_salience_with_shock_dates.png",␣
 ↪dpi=150)
plt.show()
print("Saved:", FIGURES_DIR / "monthly_volume_salience_with_shock_dates.png")
```




```
Saved: c:\PythonProjects\AfD-
project\figures\01_notebook_figures\monthly_volume_salience_with_shock_dates.png
```

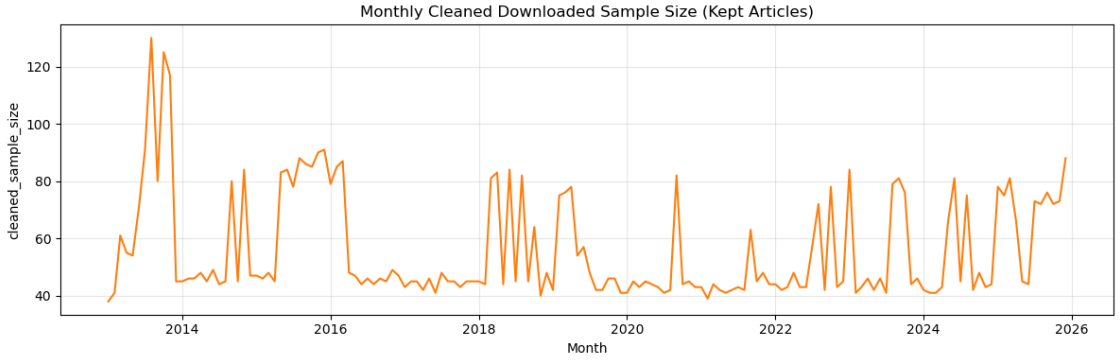
[8]:
```
plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol["media_salience_volume_z"], linewidth=1.5)
plt.axhline(0.0, linestyle="--", alpha=0.7)
plt.title("Monthly Nexis Volume Salience (Z-Score, Population SD)")
plt.xlabel("Month")
plt.ylabel("media_salience_volume_z")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(FIGURES_DIR / "monthly_volume_salience_z_report.png", dpi=150)
plt.show()
print("Saved:", FIGURES_DIR / "monthly_volume_salience_z_report.png")
```




```
Saved: c:\PythonProjects\AfD-
```

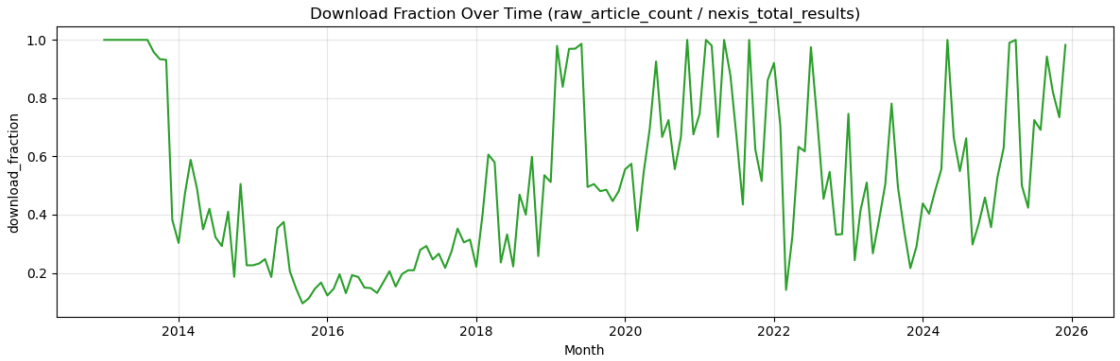
project\figures\01_notebook_figures\monthly_volume_salience_z_report.png

[9]:
```
plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol["cleaned_sample_size"], linewidth=1.5, color="tab:
 ↪orange")
plt.title("Monthly Cleaned Downloaded Sample Size (Kept Articles)")
plt.xlabel("Month")
plt.ylabel("cleaned_sample_size")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(FIGURES_DIR / "monthly_cleaned_sample_size_over_time_report.png",␣
 ↪dpi=150)
plt.show()
print("Saved:", FIGURES_DIR / "monthly_cleaned_sample_size_over_time_report.
 ↪png")
```



Saved: c:\PythonProjects\AfD-project\figures\01_notebook_figures\monthly_cleaned_sample_size_over_time_report.png

[10]:
```
plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol["download_fraction"], linewidth=1.5, color="tab:
 ↪green")
plt.title("Download Fraction Over Time (raw_article_count /␣
 ↪nexis_total_results)")
plt.xlabel("Month")
plt.ylabel("download_fraction")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(FIGURES_DIR / "download_fraction_over_time_report.png", dpi=150)
plt.show()
print("Saved:", FIGURES_DIR / "download_fraction_over_time_report.png")
```




```
Saved: c:\PythonProjects\AfD-
project\figures\01_notebook_figures\download_fraction_over_time_report.png
```

[11]:
```
print("Top 15 months by nexis_total_results:")
display(
    vol.sort_values("nexis_total_results", ascending=False)
      [["month_id", "nexis_total_results", "media_salience_volume_z"]]
      .head(15)
)

print("Months with media_salience_volume_z >= 2.0:")
display(
    vol[vol["media_salience_volume_z"] >= 2.0]
      .sort_values("media_salience_volume_z", ascending=False)
      [["month_id", "nexis_total_results", "media_salience_volume_z"]]
)

print("Top 15 months by cleaned_sample_size (diagnostic only):")
display(
    vol.sort_values("cleaned_sample_size", ascending=False)
      [["month_id", "cleaned_sample_size", "cleaned_sample_z"]]
      .head(15)
)
```

```
Top 15 months by nexis_total_results:
```

	month_id	nexis_total_results	media_salience_volume_z
32	2015-09	1007	5.450985
33	2015-10	891	4.686355
36	2016-01	766	3.862401
31	2015-08	687	3.341662
34	2015-11	665	3.196646
37	2016-02	659	3.157096
35	2015-12	581	2.642949
38	2016-03	496	2.082660

```
30   2015-07                 438                 1.700346
39   2016-04                 383                 1.337806
44   2016-09                 381                 1.324622
110  2022-03                 353                 1.140057
70   2018-11                 349                 1.113690
43   2016-08                 338                 1.041182
42   2016-07                 334                 1.014816

Months with media_salience_volume_z >= 2.0:

   month_id  nexis_total_results  media_salience_volume_z
32  2015-09                 1007                 5.450985
33  2015-10                  891                 4.686355
36  2016-01                  766                 3.862401
31  2015-08                  687                 3.341662
34  2015-11                  665                 3.196646
37  2016-02                  659                 3.157096
35  2015-12                  581                 2.642949
38  2016-03                  496                 2.082660

Top 15 months by cleaned_sample_size (diagnostic only):

    month_id  cleaned_sample_size  cleaned_sample_z
7    2013-08                  130          3.938528
9    2013-10                  125          3.671579
10   2013-11                  117          3.244460
6    2013-07                   91          1.856324
35   2015-12                   91          1.856324
34   2015-11                   90          1.802934
155  2025-12                   88          1.696155
31   2015-08                   88          1.696155
38   2016-03                   87          1.642765
32   2015-09                   86          1.589375
33   2015-10                   85          1.535985
37   2016-02                   85          1.535985
22   2014-11                   84          1.482595
120  2023-01                   84          1.482595
29   2015-06                   84          1.482595
```

[12]:
```
print("Diagnostic flags summary:")
print("retrieval_mismatch_flag months:", int(vol["retrieval_mismatch_flag"].
  ↪sum()))
print("high_download_sample_flag months:", int(vol["high_download_sample_flag"].
  ↪sum()))

print("Months where raw_article_count > nexis_total_results:")
display(
    vol[vol["retrieval_mismatch_flag"]]
```

```
      [["month_id", "raw_article_count", "nexis_total_results",␣
 ↪"download_fraction"]]
      .sort_values("month_id")
)

print("Months where raw_article_count > 50:")
display(
    vol[vol["high_download_sample_flag"]]
      [["month_id", "raw_article_count", "nexis_total_results",␣
 ↪"download_fraction"]]
      .sort_values("month_id")
)

print("Download and cleaned sample fractions (head):")
display(
    vol[["month_id", "download_fraction", "cleaned_sample_fraction"]].head(20)
)
```

```
Diagnostic flags summary:
retrieval_mismatch_flag months: 0
high_download_sample_flag months: 60
Months where raw_article_count > nexis_total_results:

Empty DataFrame
Columns: [month_id, raw_article_count, nexis_total_results, download_fraction]
Index: []

Months where raw_article_count > 50:
```

	month_id	raw_article_count	nexis_total_results	download_fraction
1	2013-02	55	55	1.000000
2	2013-03	67	67	1.000000
3	2013-04	63	63	1.000000
4	2013-05	64	64	1.000000
5	2013-06	83	83	1.000000
6	2013-07	104	104	1.000000
7	2013-08	146	146	1.000000
8	2013-09	91	95	0.957895
9	2013-10	141	151	0.933775
10	2013-11	136	146	0.931507
20	2014-09	89	217	0.410138
22	2014-11	95	188	0.505319
28	2015-05	91	257	0.354086
29	2015-06	96	256	0.375000
30	2015-07	90	438	0.205479
31	2015-08	100	687	0.145560
32	2015-09	96	1007	0.095333
33	2015-10	100	891	0.112233
34	2015-11	97	665	0.145865

35	2015-12	97	581	0.166954
36	2016-01	94	766	0.122715
37	2016-02	96	659	0.145675
38	2016-03	97	496	0.195565
62	2018-03	97	160	0.606250
63	2018-04	95	164	0.579268
65	2018-06	98	295	0.332203
67	2018-08	98	209	0.468900
69	2018-10	94	157	0.598726
70	2018-11	90	349	0.257880
71	2018-12	98	183	0.535519
72	2019-01	150	293	0.511945
73	2019-02	95	97	0.979381
74	2019-03	99	118	0.838983
75	2019-04	94	97	0.969072
76	2019-05	64	66	0.969697
77	2019-06	73	74	0.986486
92	2020-09	99	178	0.556180
100	2021-05	55	55	1.000000
104	2021-09	75	75	1.000000
108	2022-01	58	63	0.920635
114	2022-07	78	80	0.975000
115	2022-08	94	130	0.723077
117	2022-10	99	181	0.546961
120	2023-01	100	134	0.746269
127	2023-08	100	128	0.781250
128	2023-09	100	202	0.495050
129	2023-10	99	284	0.348592
136	2024-05	82	82	1.000000
137	2024-06	99	149	0.664430
139	2024-08	100	151	0.662252
144	2025-01	98	186	0.526882
145	2025-02	97	154	0.629870
146	2025-03	99	100	0.990000
147	2025-04	78	78	1.000000
150	2025-07	100	138	0.724638
151	2025-08	94	136	0.691176
152	2025-09	99	105	0.942857
153	2025-10	86	105	0.819048
154	2025-11	83	113	0.734513
155	2025-12	113	115	0.982609

Download and cleaned sample fractions (head):

	month_id	download_fraction	cleaned_sample_fraction
0	2013-01	1.000000	0.904762
1	2013-02	1.000000	0.745455
2	2013-03	1.000000	0.910448
3	2013-04	1.000000	0.873016

4	2013-05	1.000000	0.843750
5	2013-06	1.000000	0.855422
6	2013-07	1.000000	0.875000
7	2013-08	1.000000	0.890411
8	2013-09	0.957895	0.842105
9	2013-10	0.933775	0.827815
10	2013-11	0.931507	0.801370
11	2013-12	0.381679	0.343511
12	2014-01	0.303030	0.272727
13	2014-02	0.471698	0.433962
14	2014-03	0.588235	0.541176
15	2014-04	0.490196	0.470588
16	2014-05	0.349650	0.314685
17	2014-06	0.420168	0.411765
18	2014-07	0.322581	0.283871
19	2014-08	0.292398	0.263158

[13]:
```
indicator = vol.copy()
indicator["month"] = indicator["month_id"]
indicator["year_month"] = indicator["month_id"]
indicator["year"] = indicator["month_dt"].dt.year
indicator["month_num"] = indicator["month_dt"].dt.month

out_cols = [
    "month",
    "year_month",
    "year",
    "month_num",
    "nexis_total_results",
    "media_salience_volume_raw",
    "media_salience_volume_log1p",
    "media_salience_volume_z",
    "raw_article_count",
    "download_fraction",
    "retrieval_mismatch_flag",
    "high_download_sample_flag",
    "kept_article_count",
    "cleaned_sample_size",
    "cleaned_sample_fraction",
    "cleaned_sample_log1p",
    "cleaned_sample_z",
]

merge_ready = indicator[out_cols].sort_values("month").reset_index(drop=True)

assert len(merge_ready) == 156, f"Expected 156 rows, found {len(merge_ready)}"
```

```
assert not merge_ready["month"].duplicated().any(), "Duplicate month rows in␣
 ↪indicator"
assert merge_ready["nexis_total_results"].notna().all(), "Missing␣
 ↪nexis_total_results in indicator"

out_csv = DATA_DIR / "indicators" / "monthly_media_salience_indicator.csv"
out_parquet = DATA_DIR / "indicators" / "monthly_media_salience_indicator.
 ↪parquet"
merge_ready.to_csv(out_csv, index=False, encoding="utf-8-sig")
merge_ready.to_parquet(out_parquet, index=False)

print("Rows in merge-ready indicator:", len(merge_ready))
print("Saved:", out_csv)
print("Saved:", out_parquet)
display(merge_ready.head(12))
```

```
Rows in merge-ready indicator: 156
Saved: c:\PythonProjects\AfD-
project\data\indicators\monthly_media_salience_indicator.csv
Saved: c:\PythonProjects\AfD-
project\data\indicators\monthly_media_salience_indicator.parquet
```

	month	year_month	year	month_num	nexis_total_results	\
0	2013-01	2013-01	2013	1	42	
1	2013-02	2013-02	2013	2	55	
2	2013-03	2013-03	2013	3	67	
3	2013-04	2013-04	2013	4	63	
4	2013-05	2013-05	2013	5	64	
5	2013-06	2013-06	2013	6	83	
6	2013-07	2013-07	2013	7	104	
7	2013-08	2013-08	2013	8	146	
8	2013-09	2013-09	2013	9	95	
9	2013-10	2013-10	2013	10	151	
10	2013-11	2013-11	2013	11	146	
11	2013-12	2013-12	2013	12	131	

	media_salience_volume_raw	media_salience_volume_log1p	\
0	42	3.761200	
1	55	4.025352	
2	67	4.219508	
3	63	4.158883	
4	64	4.174387	
5	83	4.430817	
6	104	4.653960	
7	146	4.990433	
8	95	4.564348	
9	151	5.023881	
10	146	4.990433	

```
11                      131                    4.882802

    media_salience_volume_z  raw_article_count  download_fraction  \
0                 -0.909941                 42           1.000000
1                 -0.824250                 55           1.000000
2                 -0.745150                 67           1.000000
3                 -0.771517                 63           1.000000
4                 -0.764925                 64           1.000000
5                 -0.639684                 83           1.000000
6                 -0.501260                104           1.000000
7                 -0.224411                146           1.000000
8                 -0.560585                 91           0.957895
9                 -0.191453                141           0.933775
10                -0.224411                136           0.931507
11                -0.323286                 50           0.381679

    retrieval_mismatch_flag  high_download_sample_flag  kept_article_count  \
0                     False                      False                  38
1                     False                       True                  41
2                     False                       True                  61
3                     False                       True                  55
4                     False                       True                  54
5                     False                       True                  71
6                     False                       True                  91
7                     False                       True                 130
8                     False                       True                  80
9                     False                       True                 125
10                    False                       True                 117
11                    False                      False                  45

    cleaned_sample_size  cleaned_sample_fraction  cleaned_sample_log1p  \
0                    38                 0.904762              3.663562
1                    41                 0.745455              3.737670
2                    61                 0.910448              4.127134
3                    55                 0.873016              4.025352
4                    54                 0.843750              4.007333
5                    71                 0.855422              4.276666
6                    91                 0.875000              4.521789
7                   130                 0.890411              4.875197
8                    80                 0.842105              4.394449
9                   125                 0.827815              4.836282
10                  117                 0.801370              4.770685
11                   45                 0.343511              3.828641

    cleaned_sample_z
0          -0.973338
1          -0.813169
2           0.254629
```

```
3          -0.065711
4          -0.119100
5           0.788527
6           1.856324
7           3.938528
8           1.269036
9           3.671579
10          3.244460
11         -0.599609
```

1.1 Interpretation Notes

- Main monthly media salience indicator: `nexis_total_results`.
- `raw_article_count` reflects downloaded sample size / retrieval effort.
- `kept_article_count` and `cleaned_sample_size` reflect cleaned downloaded text sample size.
- Downloaded and cleaned samples remain the analytical text corpus for source composition and text diagnostics.
- Absolute month-level salience volume comes from Nexis total monthly results because download intensity varied across months.
- No polling merge, causal model, sentiment modeling, topic modeling, or event-study estimation is added in this notebook.